

Microsoft Foundry 中的检索增强生成 (RAG) 与索引

原文: Retrieval augmented generation (RAG) and indexes in Microsoft Foundry

来源: <https://learn.microsoft.com/en-au/azure/foundry/concepts/retrieval-augmented-generation>

原文更新时间: 2026-05-18

许可证: Creative Commons Attribution 4.0 International

说明: 本文是对 Microsoft Learn 原文的中文翻译与术语本地化整理, 保留原文结构和主要含义。代码、产品名、API 名称和链接按原文保留。

概览

检索增强生成 (retrieval augmented generation, RAG) 是一种把搜索能力和大型语言模型 (LLM) 结合起来的模式。它的目标是让模型回答时基于你的数据, 而不是只依赖模型训练时学到的公共知识。本文解释 RAG 在 Microsoft Foundry 中如何工作、索引在其中承担什么角色, 以及 agentic retrieval 如何改变传统 RAG 模式。

LLM 通常基于训练时可用的公共数据训练。如果你需要模型基于私有数据回答, 或者需要基于经常变化的信息回答, RAG 可以帮助你:

- 从你的数据中检索相关信息, 通常通过索引完成。
- 把这些信息作为 grounding data 提供给模型。
- 生成可以引用原始内容来源的回答。

什么是 RAG

像 ChatGPT 这样的大型语言模型, 是基于模型训练时可用的公共互联网数据训练出来的。这些公共数据并不总能满足你的业务需求。例如, 你可能希望回答基于企业内部文档, 或者希望答案反映最新信息。

RAG 的做法是从你的数据中检索相关内容, 并把这些内容放进模型输入。这样, 模型生成回答时就可以基于检索到的内容, 而不是凭空推断。

RAG 的几个关键概念:

- **Grounding data:** 提供给模型的检索内容, 用来减少猜测。
- **Index:** 为检索优化的数据结构, 可以支持关键词、语义、向量或混合搜索。
- **Embeddings:** 内容的数值化表示, 用于向量相似度搜索。可参考 Microsoft Learn 的 Understand embeddings。

- **System message and prompts:** 指导模型如何使用检索内容的指令。可参考 Prompt engineering 和 Safety system messages。

RAG 如何工作

RAG 通常遵循三步流程：

1. **Retrieve:** 用户提出问题后，应用查询索引或数据存储，找出相关内容。
2. **Augment:** 应用把用户问题和检索内容，也就是 grounding data，组合成一个 prompt。
3. **Generate:** 模型接收增强后的 prompt，生成基于检索内容的回答，从而减少不准确内容，并支持更准确的引用。

这个流程的核心不是让模型“记住”所有私有知识，而是在回答时动态取回相关证据，再让模型围绕证据生成答案。

什么是索引，为什么需要它

RAG 的效果很大程度取决于能否快速、稳定地取回相关内容。索引通过组织内容来提升检索效率。

很多 RAG 方案会使用支持一种或多种检索模式的索引：

- **Keyword search:** 关键词搜索。
- **Semantic search:** 语义搜索。
- **Vector search:** 向量搜索。
- **Hybrid search:** 混合搜索，通常指关键词搜索加向量搜索，有时还会配合语义排序。

索引也可以保存提升引用质量所需的字段，例如文档标题、URL、文件名等。

Foundry 可以把项目连接到 Azure AI Search 服务和索引，用于检索。根据所使用的功能和 API 表面不同，这些连接信息可能表现为项目连接，也可能表现为 index asset ID。

例如，Foundry Project REST API preview 对 Azure AI Search 索引资源包含 `index_asset_id` 字段。可参考 Foundry Project REST API preview。

Azure AI Search 是 RAG 场景推荐使用的索引存储。它支持对搜索索引中的向量数据和文本数据进行检索；如果使用 agentic retrieval，也可以查询其他目标。可参考 What is Azure AI Search。

Agentic RAG：现代检索方式

传统 RAG 往往使用单个查询从数据中检索信息。Agentic retrieval，也叫 agentic RAG，是检索架构的一种演进。它使用模型把复杂输入拆分成多个聚焦的子查询，并行执行这些子查询，然后返回更适合聊天补全模型使用的结构化 grounding data。

与经典 RAG 相比，agentic retrieval 有几类优势：

- **上下文感知的查询规划**：利用对话历史理解上下文和意图。追问可以保留前文上下文，让多轮对话更自然。
- **并行执行**：同时运行多个聚焦子查询，提升覆盖面。相比只用单个查询顺序检索，并行执行可以降低延迟，并取回更多样的相关结果。
- **结构化响应**：随结果一起返回 grounding data、引用和执行元数据。结构化输出让应用更容易准确引用来源，并追踪答案背后的检索过程。
- **内置语义排序**：提升结果相关性。语义排序可以过滤噪声，优先保留真正相关的段落，这在大型数据集上尤其重要。
- **可选答案合成**：查询响应中可以直接包含由 LLM 组织的答案。你也可以选择只返回原始段落，让应用自己处理。

如果你使用 Azure AI Search 作为检索引擎，可参考 [Agentic retrieval](#) 和 [Quickstart: Agentic retrieval](#)。

在 Foundry 中如何选择方案

Foundry 支持多种处理私有数据的模式。选择时应根据用例复杂度，以及你需要多少控制权来决定。

- **使用 RAG**：当你需要答案基于私有数据或频繁变化的数据时。
- **使用 fine-tuning**：当你要改变模型行为、风格或任务表现，而不是添加新知识时。
- **使用 agent tools**：当你正在构建一个需要把检索作为工具来使用的 agent 时。例如可参考 [File search tool for agents](#)。

换句话说，RAG 更适合“知识从外部取回”，fine-tuning 更适合“改变模型做事方式”，agent tools 更适合“让 agent 在任务执行中调用检索能力”。

在 Foundry 中开始使用 RAG

在 Foundry 中实现 RAG，通常会遵循下面的工作流：

1. **准备数据**：整理并切分你的私有文档或知识库，使其成为可搜索内容。
2. **设置索引**：创建 Azure AI Search 索引，或使用其他检索服务来组织内容，以便高效搜索。
3. **连接到 Foundry**：从 Foundry 项目创建到索引或检索服务的连接。
4. **构建 RAG 应用**：使用 Foundry SDK 或 REST API，把检索过程和 LLM 调用集成起来。
5. **测试与评估**：验证检索质量是否足够好，回答是否准确，引用是否恰当。

开始时，可以按需求选择路径：

- **Agent with retrieval**：如果你正在构建 agent，把检索作为工具使用。可参考 [File search tool for agents](#)。
- **Custom RAG application**：如果你需要完整控制权，用 Foundry SDK 构建完整 RAG 应用。

安全与隐私注意事项

如果访问控制和 prompt 设计不当，RAG 系统可能暴露敏感内容。

- **在检索时应用访问控制。**如果你使用 Azure AI Search 作为数据源，可以用文档级访问控制和安全过滤器。
- **生产环境优先使用 Microsoft Entra ID，而不是 API keys。**API keys 适合开发阶段，但不推荐用于生产环境。Azure AI Search 的 RBAC 指南可参考 [Connect to Azure AI Search using roles](#)。
- **把检索到的内容视为不可信输入。**系统消息和应用逻辑应降低文档或检索段落中的 prompt injection 风险。可参考 [Safety system messages](#)。

成本与延迟注意事项

与只调用模型相比，RAG 会增加额外工作：

- **检索成本和延迟：**查询索引会增加网络往返和计算。
- **Embedding 成本和延迟：**向量搜索需要在建索引时生成 embedding，很多场景下查询时也需要生成 embedding。
- **Token 使用量：**检索段落会增加输入 token，从而增加成本。

如果使用 Azure AI Search，在生产上线前应确认服务层级和价格。如果使用语义检索或混合检索，应查看 Azure AI Search 文档中的价格和限制。

限制与故障排查

已知限制

- RAG 质量依赖内容准备、检索配置和 prompt 设计。糟糕的数据准备或索引策略会直接影响回答质量。
- 如果检索返回的段落不相关或不完整，即使有 grounding，模型仍可能生成不完整或不准确的回答。
- 如果没有控制源内容访问权限，基于检索内容生成的回答可能泄露索引中的敏感信息。

常见挑战与缓解方式

- **检索质量差：**如果索引没有返回相关段落，应检查数据切分策略、embedding 模型质量和搜索配置，例如关键词、语义、向量或混合检索的选择。
- **有 grounding 仍然幻觉：**即便提供了检索内容，模型仍可能生成不准确回答。应启用引用，并使用清晰的 system message 和 prompt，要求模型严格基于检索内容回答。
- **延迟问题：**大型索引可能拖慢检索。可以优化索引策略、过滤条件和 reranking，减少需要处理的段落数量。
- **超出 token 预算：**检索段落可能很快耗尽 token 限制。应实现段落过滤、排序或摘要，确保输入控制在预算内。

评估 RAG 效果的更多指导，可以参考原文 Related content 中列出的教程和 quickstarts。

相关内容

- File search tool for agents
- Quickstart: Agentic retrieval
- Agentic retrieval overview
- What is Azure AI Search

译者整理的实践结论

从这篇文档可以提炼出一个清晰建议：

- 企业知识问答的默认底座应是 RAG，而不是 fine-tuning。
- 索引不是可选配件，而是 RAG 稳定工作的核心基础设施。
- 生产系统应优先考虑混合检索、引用、权限过滤、评估和延迟成本。
- Agentic RAG 适合复杂问题、多轮问题、多子查询和需要更强检索编排的场景。
- Agent 工具适合“agent 在任务中调用检索”，但知识问答本身仍应基于可追溯的检索证据。